

Correction TP 1 : Jeux de données et variables

HAMLIL Mohamed

2025-01-15

Table des matières

1	Exercice 0 : Prise en main	2
1.1	1. Installer un package	2
1.2	2. Charger un package	3
1.3	3. Documentation du package	3
1.4	4. Variables	3
1.5	5. Types de données dans R	4
1.6	6. Opérations	5
1.7	7. Vecteurs	6
1.8	8. Quelques fonctions usuelles	7
1.9	9. Matrices	8
1.10	10. Listes	8
1.11	11. Données manquantes et valeurs spéciales	9
1.12	12. Écrire une fonction	9
1.13	13. Data Frames	9
1.14	14. Charger un jeu de données	10
1.15	15. Fonctions graphiques de base	10
2	Exercice 1 : Analyse préliminaire d'un jeu de données	11
2.1	1. Charger et inspecter les données	11
2.1.1	Transformation de variables	12
2.2	2. Analyse statistique univariée	12
2.3	3. Analyse graphique univariée	14
2.4	4. Analyse multivariée	15
2.4.1	Quantitative vs Qualitative (<code>diabetes</code> vs <code>test</code>)	16
2.4.2	Deux variables qualitatives (<code>age_cat</code> vs <code>test</code>)	17

1 Exercice 0 : Prise en main

J'aime utiliser **RStudio**, une interface du logiciel R qui est bien pratique car elle comprend :

- Une fenêtre contenant notre script R.
- Une fenêtre console pour exécuter des commandes (comme dans R).
- Une fenêtre contenant les onglets Environment (environnement).
- Une fenêtre contenant les onglets Plots (graphiques), Packages, Help (aide).

Installation

Pour installer **RStudio** c'est [ici](#).

R Markdown (et **Quarto**) permet de créer un document avec du texte, des extraits de code R et leurs résultats, et d'obtenir un beau PDF.

[Plus d'informations ici](#).

1.1 1. Installer un package

Le *Comprehensive R Archive Network* (CRAN) est le répertoire officiel de packages R. Avant de pouvoir utiliser un package, il faut s'assurer qu'il soit installé sur notre machine.

Pour installer un package, on utilise la commande `install.packages`.

Avertissement

Attention aux guillemets ! Sinon, R cherchera une variable nommée `tinytex` au lieu du package.

```
install.packages("tinytex")  
install.packages("rmarkdown")
```

À faire

Installer le package `faraway` que l'on trouve dans CRAN.

```
install.packages("faraway")
```

*Note : On utilise `eval=FALSE` dans le **R Markdown** pour ne pas réinstaller le package à chaque compilation du PDF.*

1.2 2. Charger un package

Une fois qu'un package est installé, il faut le charger avec la commande `library`.

```
library(faraway)
```

i À faire

Installer et charger le package `ggplot2`.

```
install.packages("ggplot2")
```

```
library(ggplot2)
```

1.3 3. Documentation du package

On accède à la documentation via l'onglet *Packages* ou directement via la commande `?`. Par exemple, pour le jeu de données `pima` du package `faraway` :

```
?pima
```

i À faire

Accéder à la documentation du jeu de données `beetle`.

1.4 4. Variables

Il est recommandé de commencer avec un environnement propre.

```
rm(list = ls())
```

Les variables stockent des données. Règles de nommage :

1. Ne peut pas commencer par un chiffre.
2. Uniquement caractères alphanumériques et underscores (`_`) ou points (`.`).
3. Sensible à la casse (`az9_8` `Az9_8`).

! Important

Attention à ne pas utiliser des noms réservés (comme `mean`, `c`, `data`) pour éviter des conflits.

Assignment : On utilise `<-` (préférable en R) ou `=`.

```
az9_8 = 7
```

Gestion :

- `ls()` : Lister les objets.
- `rm()` : Supprimer un objet.

```
ls()
```

```
[1] "az9_8"
```

```
rm(az9_8)
```

1.5 5. Types de données dans R

Les types principaux sont :

- `numeric` : nombres réels.
- `integer` : entiers (ex : `2L`).
- `character` : texte.
- `logical` : booléen (`TRUE`, `FALSE`).
- `factor` : variable catégorielle.

```
class(2)
```

```
[1] "numeric"
```

```
class(2L)
```

```
[1] "integer"
```

```
class("texte")
```

```
[1] "character"
```

```
class(TRUE)
```

```
[1] "logical"
```

1.6 6. Opérations

Séquences : L'opérateur `:` génère une séquence d'entiers.

```
1:50
```

```
[1]  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25
[26] 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50
```

Arithmétique : `+`, `-`, `*`, `/`, `^` (puissance). **Assignment** : `=` ou `<-`.

```
x = 2
y = (-1)^2
z = 10 / 3
w = "lapin"
class(w)
```

```
[1] "character"
```

Comparaison : `==` (égalité), `!=` (différence), `>`, `<`, `>=`, `<=`.

```
x != y
```

```
[1] TRUE
```

Logique et Contrôle : `&` (ET), `|` (OU), `!` (NON), `if`, `else`, `for`, `while`.

```
if(z < 2){
  z = "petit"
} else {
  z = "grand"
}
z
```

```
[1] "grand"
```

Accès :

- [] : accès par index (vecteurs, matrices).
- \$: accès par nom (listes, data frames).

1.7 7. Vecteurs

On crée un vecteur avec `c()` (combine).

```
v = c(1, 2, 4)
w = 100:150      # Séquence
z = rep("souris", times = 5) # Répétition (réduit à 5 pour l'affichage)
y = seq(from = 0, to = 1, by = 0.1) # Séquence avec pas
```

Indexation : Les indices commencent à **1**.

```
v[3]      # 3ème élément
```

```
[1] 4
```

```
v[-2]     # Tout sauf le 2ème
```

```
[1] 1 4
```

```
w[c(1, 10)] # 1er et 10ème éléments
```

```
[1] 100 109
```

Conversion de type :

```
v_char = c("a","b","c","a","c","c")
w_fact = as.factor(v_char) # Conversion en facteur
levels(w_fact)
```

```
[1] "a" "b" "c"
```

1.8 8. Quelques fonctions usuelles

Lois de probabilité :

- `r...` (random) : génération aléatoire.
- `d...` (density) : densité.
- `p...` (probability) : fonction de répartition.
- `q...` (quantile) : quantiles.

```
# Echantillonnage
x = runif(n = 10, min = 0, max = 1)
y = rbinom(n = 10, size = 1, prob = 0.5)

# Tirage avec/sans remise
sample(x = 1:10, size = 5, replace = FALSE)
```

```
[1] 8 10 3 7 5
```

Statistiques descriptives :

```
x = rnorm(n = 1000, mean = 5, sd = 2)
mean(x)
```

```
[1] 4.975829
```

```
var(x)
```

```
[1] 4.012303
```

```
summary(x)
```

```
      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-1.155    3.645    4.934    4.976    6.337   11.648
```

Tri : `sort()` (trie les valeurs) et `order()` (donne les indices).

```
v = c(1, 10, 3, 8, -5)
sort(v, decreasing = TRUE)
```

```
[1] 10  8  3  1 -5
```

```
order(v)
```

```
[1] 5 1 3 4 2
```

1.9 9. Matrices

Création avec `matrix()`.

```
M = matrix(c(1,2,3,4,5,6), nrow = 2, ncol = 3, byrow = TRUE)
P = matrix(c(1,2,3,4,5,6), nrow = 2, ncol = 3, byrow = FALSE)
```

Opérations :

- `dim()`, `nrow()`, `ncol()` : dimensions.
- `t()` : transposée.
- `%*%` : produit matriciel.
- `solve()` : inverse.
- `diag()` : diagonale.

Exercice

Créer une matrice carrée P (taille 5) avec des tirages Bernoulli (). Créer Q diagonale contenant la diagonale de P .

```
P = matrix(rbinom(n = 25, size = 1, prob = 0.6), nrow = 5, ncol = 5)
Q = diag(diag(P))
```

1.10 10. Listes

Les listes peuvent contenir des objets de types différents.


```
l = list(vecteur = 1:10, noms = c("lapin","souris"))
l$noms # Accès par nom
```

```
[1] "lapin" "souris"
```

1.11 11. Données manquantes et valeurs spéciales

- NA : Not Available (manquant).
- NaN : Not a Number (0/0).
- Inf : Infini.

```
x = c(1, 2, NA)
is.na(x)
```

```
[1] FALSE FALSE TRUE
```

1.12 12. Écrire une fonction

```
g = function(x, y = 2){ # y a une valeur par défaut de 2
  z = x + y
  return(list(res_x = x, res_z = z))
}

# Appel de la fonction
g(x = 1) # utilise y = 2 par défaut
```

```
$res_x
[1] 1
```

```
$res_z
[1] 3
```

1.13 13. Data Frames

Structure classique des données : colonnes = variables, lignes = individus.

```
d = data.frame(  
  x = rep(1, 10),  
  y = 1:10,  
  char = letters[1:10]  
)  
head(d)
```

```
  x y char  
1 1 1    a  
2 1 2    b  
3 1 3    c  
4 1 4    d  
5 1 5    e  
6 1 6    f
```

1.14 14. Charger un jeu de données

- Via un package : `data(pima)` (package `faraway`).
- Via un fichier : `read.csv()`.

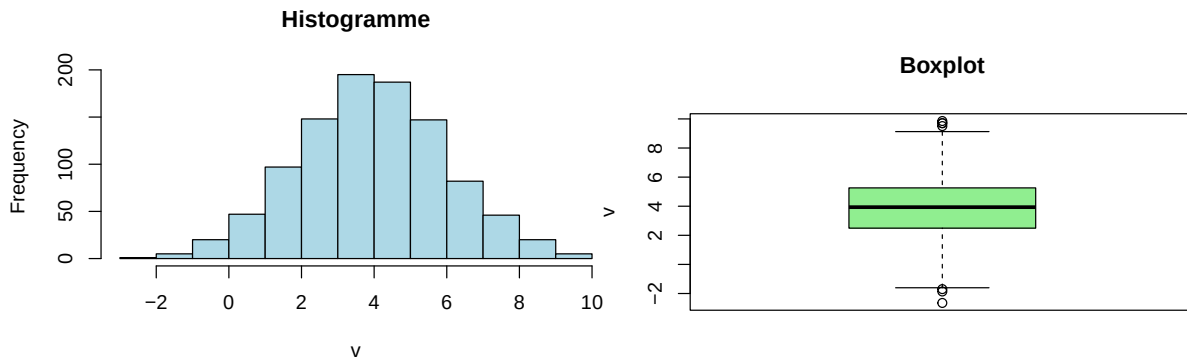
```
# Exemple si le fichier est dans le dossier parent  
data = read.csv("../titanic.csv")
```

1.15 15. Fonctions graphiques de base

```
v = rnorm(1000, mean = 4, sd = 2)  
  
# Histogramme  
hist(v, col = "lightblue", main = "Histogramme", xlab = "v")  
# Boxplot  
boxplot(v, col = 'lightgreen', main = "Boxplot", ylab = "v")
```

Paramètres graphiques :

- `par(mfrow = c(1,2))` : divise la fenêtre en 1 ligne, 2 colonnes.
- `ylim`, `xlim` : limites des axes.
- `main`, `xlab`, `ylab` : titres et labels.



2 Exercice 1 : Analyse préliminaire d'un jeu de données

2.1 1. Charger et inspecter les données

Le jeu donnée `pima` est dans le package `faraway`.

```
data(pima)
head(pima)
```

	pregnant	glucose	diastolic	triceps	insulin	bmi	diabetes	age	test
1	6	148	72	35	0	33.6	0.627	50	1
2	1	85	66	29	0	26.6	0.351	31	0
3	8	183	64	0	0	23.3	0.672	32	1
4	1	89	66	23	94	28.1	0.167	21	0
5	0	137	40	35	168	43.1	2.288	33	1
6	5	116	74	0	0	25.6	0.201	30	0

Dimensions :

```
nrow(pima) # 768 lignes
```

```
[1] 768
```

```
ncol(pima) # 9 colonnes
```

```
[1] 9
```

Types de variables :

```
str(pima) # Donne une vue d'ensemble compacte des types
```

```
'data.frame': 768 obs. of 9 variables:
 $ pregnant : int 6 1 8 1 0 5 3 10 2 8 ...
 $ glucose : int 148 85 183 89 137 116 78 115 197 125 ...
 $ diastolic: int 72 66 64 66 40 74 50 0 70 96 ...
 $ triceps : int 35 29 0 23 35 0 32 0 45 0 ...
 $ insulin : int 0 0 0 94 168 0 88 0 543 0 ...
 $ bmi : num 33.6 26.6 23.3 28.1 43.1 25.6 31 35.3 30.5 0 ...
 $ diabetes : num 0.627 0.351 0.672 0.167 2.288 ...
 $ age : int 50 31 32 21 33 30 26 29 53 54 ...
 $ test : int 1 0 1 0 1 0 1 0 1 1 ...
```

2.1.1 Transformation de variables

Variable catégorielle : age. Plusieurs stratégies de discrétisation :

1. Intervalles de même largeur (cut).
2. Intervalles avec même effectif (quantiles).
3. Groupes sémantiques (Enfant, Adulte, Senior).

Application (Groupes sémantiques) :

```
pima$age_cat = cut(pima$age,
                   breaks = c(21, 30, 55, 65, 81),
                   include.lowest = TRUE)
```

2.2 2. Analyse statistique univariée

Pour les variables **quantitatives** : moyenne, variance, quartiles. Pour les variables **qualitatives** : tables de contingence.

```
summary(pima)
```

pregnant		glucose		diastolic		triceps	
Min.	: 0.000	Min.	: 0.0	Min.	: 0.00	Min.	: 0.00
1st Qu.:	1.000	1st Qu.:	99.0	1st Qu.:	62.00	1st Qu.:	0.00
Median	: 3.000	Median	:117.0	Median	: 72.00	Median	:23.00
Mean	: 3.845	Mean	:120.9	Mean	: 69.11	Mean	:20.54

3rd Qu.: 6.000	3rd Qu.:140.2	3rd Qu.: 80.00	3rd Qu.:32.00
Max. :17.000	Max. :199.0	Max. :122.00	Max. :99.00
insulin	bmi	diabetes	age
Min. : 0.0	Min. : 0.00	Min. :0.0780	Min. :21.00
1st Qu.: 0.0	1st Qu.:27.30	1st Qu.:0.2437	1st Qu.:24.00
Median : 30.5	Median :32.00	Median :0.3725	Median :29.00
Mean : 79.8	Mean :31.99	Mean :0.4719	Mean :33.24
3rd Qu.:127.2	3rd Qu.:36.60	3rd Qu.:0.6262	3rd Qu.:41.00
Max. :846.0	Max. :67.10	Max. :2.4200	Max. :81.00
test	age_cat		
Min. :0.000	[21,30]:417		
1st Qu.:0.000	(30,55]:301		
Median :0.000	(55,65]: 37		
Mean :0.349	(65,81]: 13		
3rd Qu.:1.000			
Max. :1.000			

Avertissement

Anomalies détectées

- `test` est binaire (0/1) mais vue comme numérique.
- `diastolic` (pression sanguine) a des valeurs à 0, ce qui est impossible physiologiquement. Cela indique des données manquantes.

Correction des données :

```
# Conversion en facteur
pima$test = as.factor(pima$test)
levels(pima$test) = c("négatif","positif")

# Remplacement des 0 par NA
vars_to_fix = c("diastolic", "glucose", "triceps", "insulin", "bmi")
for(v in vars_to_fix){
  pima[pima[[v]] == 0, v] = NA
}

summary(pima)
```

pregnant	glucose	diastolic	triceps
Min. : 0.000	Min. : 44.0	Min. : 24.00	Min. : 7.00
1st Qu.: 1.000	1st Qu.: 99.0	1st Qu.: 64.00	1st Qu.:22.00
Median : 3.000	Median :117.0	Median : 72.00	Median :29.00

Mean : 3.845	Mean :121.7	Mean : 72.41	Mean :29.15
3rd Qu.: 6.000	3rd Qu.:141.0	3rd Qu.: 80.00	3rd Qu.:36.00
Max. :17.000	Max. :199.0	Max. :122.00	Max. :99.00
	NA's :5	NA's :35	NA's :227
insulin	bmi	diabetes	age
Min. : 14.00	Min. :18.20	Min. :0.0780	Min. :21.00
1st Qu.: 76.25	1st Qu.:27.50	1st Qu.:0.2437	1st Qu.:24.00
Median :125.00	Median :32.30	Median :0.3725	Median :29.00
Mean :155.55	Mean :32.46	Mean :0.4719	Mean :33.24
3rd Qu.:190.00	3rd Qu.:36.60	3rd Qu.:0.6262	3rd Qu.:41.00
Max. :846.00	Max. :67.10	Max. :2.4200	Max. :81.00
NA's :374	NA's :11		
test	age_cat		
négatif:500	[21,30]:417		
positif:268	(30,55]:301		
	(55,65]: 37		
	(65,81]: 13		

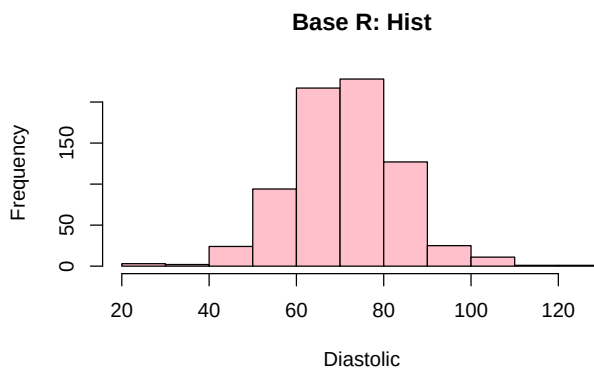
2.3 3. Analyse graphique univariée

Variable Quantitative : Pression sanguine (diastolic)

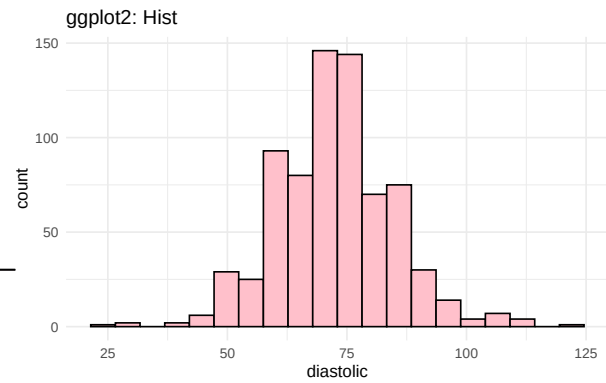
```
# Base R
hist(pima$diastolic, col = "pink", main = "Base R: Hist", xlab = "Diastolic")
# ggplot2
ggplot(pima, aes(x = diastolic)) +
  geom_histogram(fill = "pink", color = "black", bins = 20, na.rm = TRUE) +
  labs(title = "ggplot2: Hist") + theme_minimal()
```

Variable Qualitative : Age catégorisé

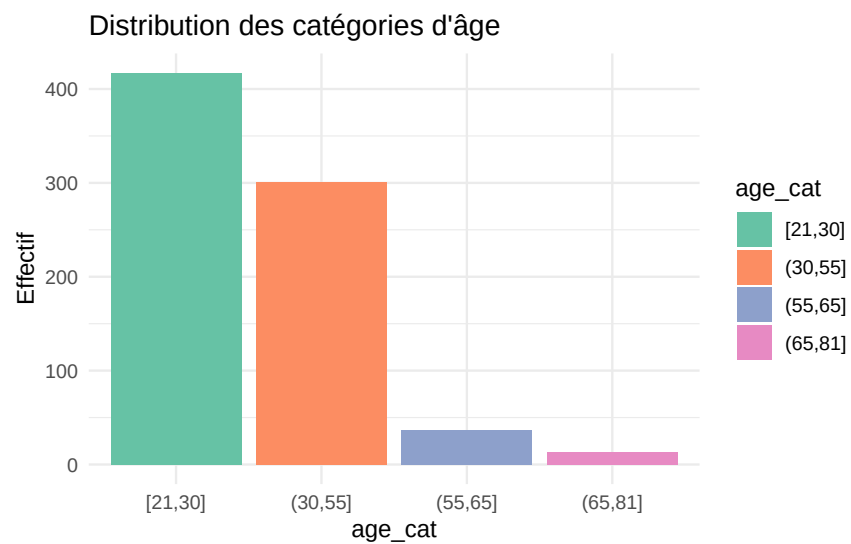
```
ggplot(pima, aes(x = age_cat, fill = age_cat)) +
  geom_bar() +
  scale_fill_brewer(palette = "Set2") +
  labs(title = "Distribution des catégories d'âge", y = "Effectif") +
  theme_minimal()
```



(a) Distribution de la pression diastolique



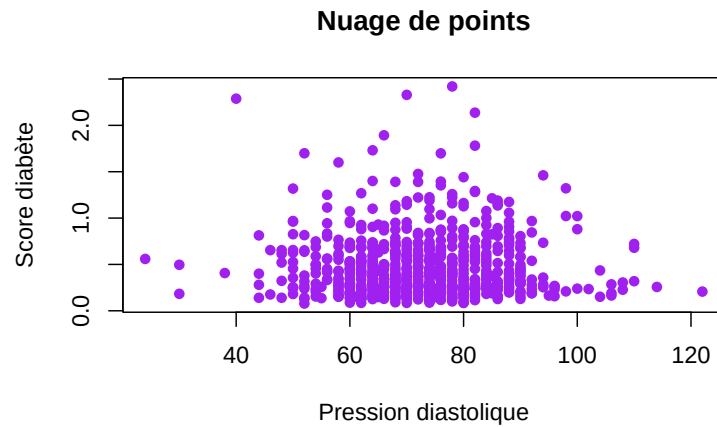
(a) Distribution de la pression diastolique



2.4 4. Analyse multivariée

Deux variables quantitatives (diabetes vs diastolic)

```
plot(pima$diastolic, pima$diabetes, col = "purple", pch = 16,
     xlab = "Pression diastolique", ylab = "Score diabète",
     main = "Nuage de points")
```



Pour voir toutes les relations 2 à 2 : `pairs()`

2.4.1 Quantitative vs Qualitative (diabetes vs test)

On utilise des **boxplots**.

```
ggplot(pima, aes(x = test, y = diabetes, fill = test)) +
  geom_boxplot() +
  scale_fill_manual(values = c("pink", "lightblue")) +
  labs(title = "Score diabète selon le résultat du test") +
  theme_minimal()
```



2.4.2 Deux variables qualitatives (age_cat vs test)

On utilise des **barplots empilés**.

Préparation des données avec `reshape2` ou `dplyr` (plus moderne) :

```
# Barplot proportionnel (position = "fill")
ggplot(pima, aes(x = age_cat, fill = test)) +
  geom_bar(position = "fill", color = "black") +
  scale_fill_brewer(palette = "Pastel1") +
  labs(y = "Proportion", title = "Proportion de tests positifs par âge") +
  theme_minimal()
```

